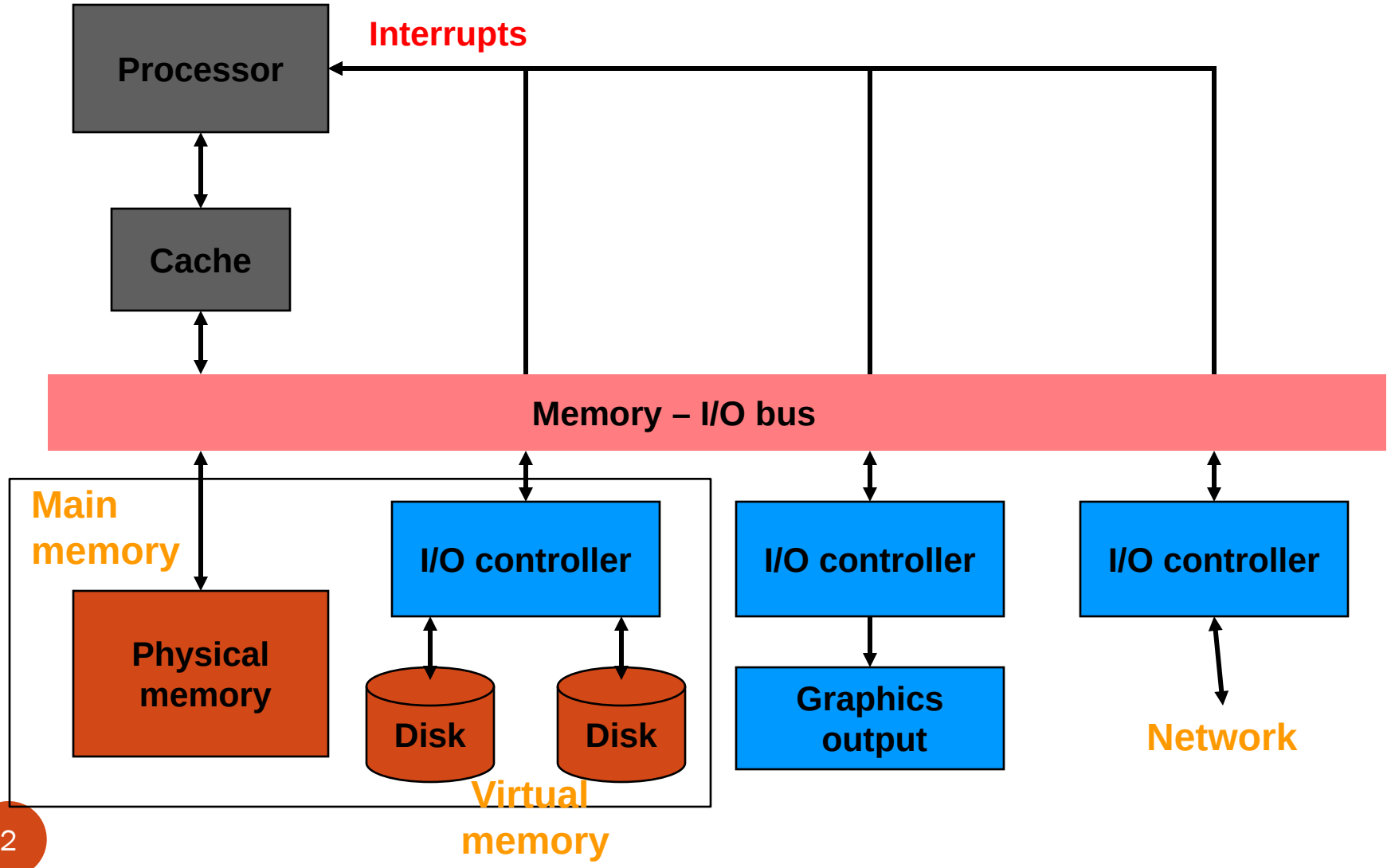
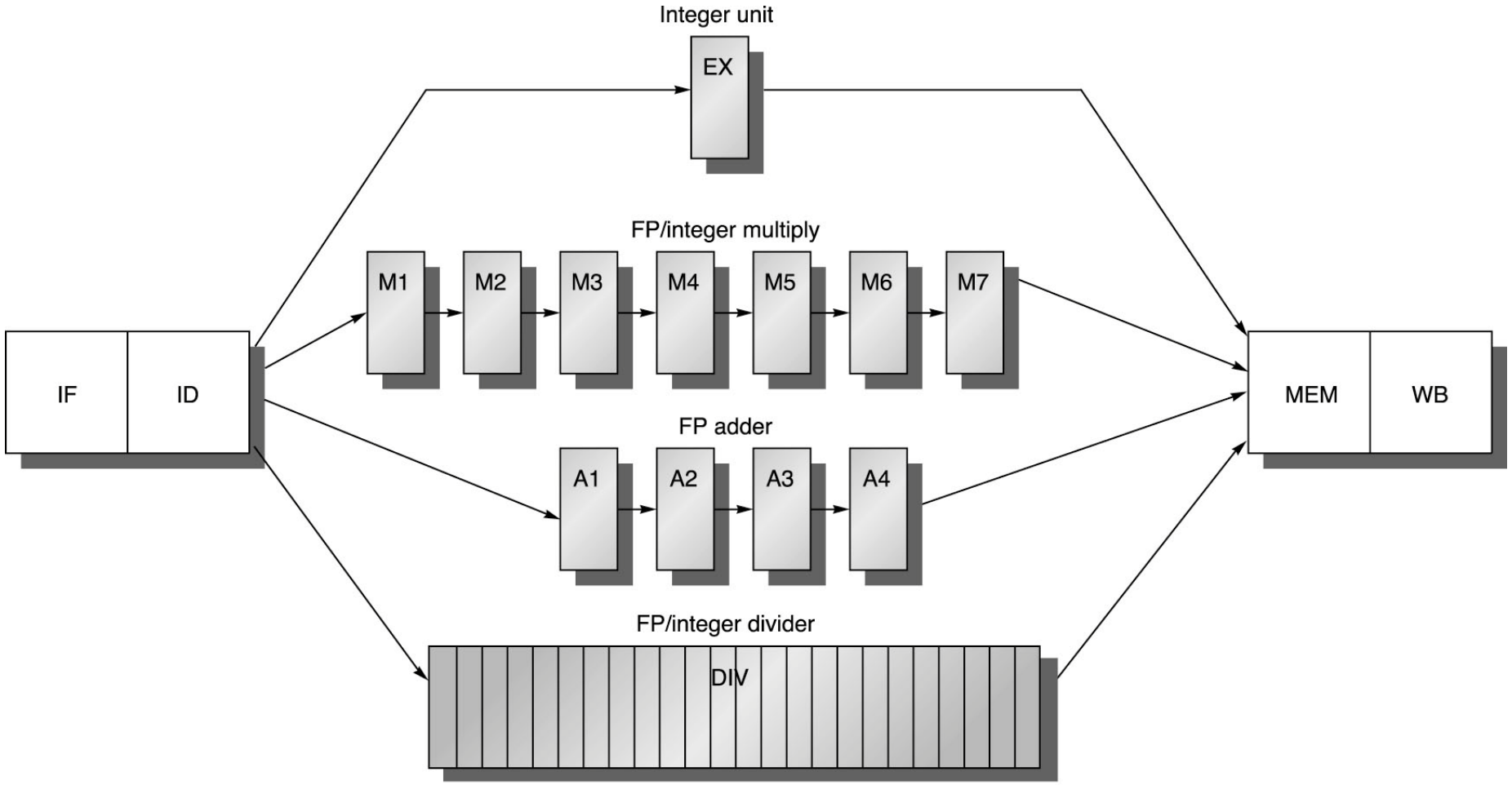
Instruction Level Parallelism (ILP)

(Micro-Architecture)

A Computer System



MIPS R4000 Pipeline



Topics in Computer Architecture

- Instruction set
- Registers, Memory Map, Addressing modes
- Binary arithmetic (2's complement, IEEE 754 floating point standard, addition, multiplication, division), from DLD
- Input and output
- Memory Hierarchy (Virtual memory, Main Memory, Cache)
- Datapaths (single-cycle, multicycle, pipeline)
- Control (combinational logic, FSM, microcode)
- Pipelining (throughput, hazards, forwarding, stall, branch prediction)
- Advanced architectures (ILP, OOE, superscalar, etc.)
- Multiprocessors, Parallel Processing
- GPGPUs
- Performance (benchmarks, energy efficiency, Amdal's law)
- Not discussed in this course:
  - Power management

Advanced Architectural Concepts

- Can we achieve CPI < 1?
  (i.e., can we have IPC > 1?)

  State-of-the-Art Microprocessor

- "Superscalar" execution or Instruction Level Parallelism (ILP)

+ "Deeper Pipeline => Dynamic Branch Prediction => Speculation => Recovery

- "Out-of-order" (OOE) Execution => Instruction Buffer and Prefetch => Reorder Buffers
- "VLIW" Very Long Instruction Word Ex: Intel/HP Titanium

Instruction Parallelism (ILP)

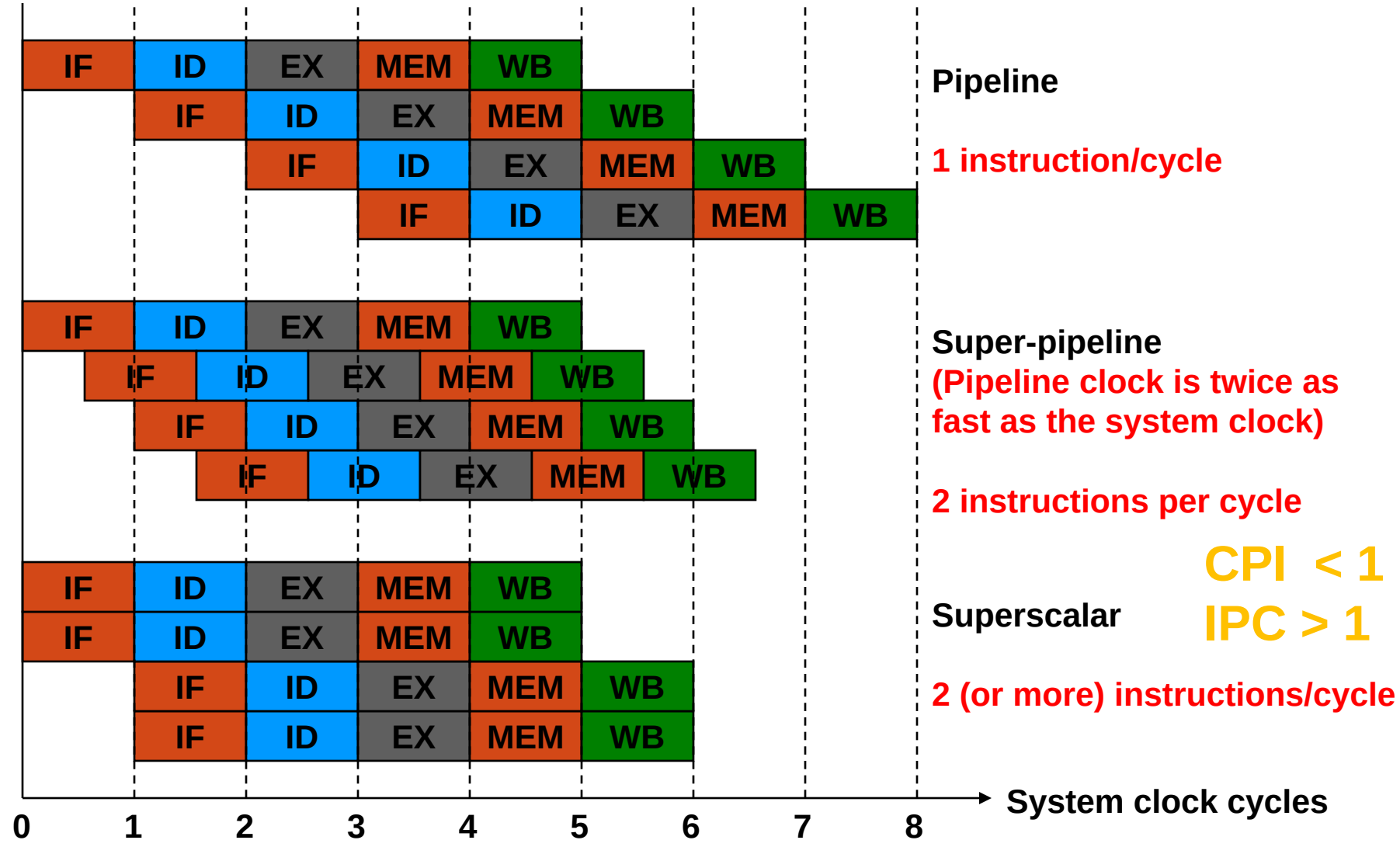
- Key idea: Allow instructions behind stall to proceed
- Instruction level parallelism (ILP): multiple instructions fetched and executed simultaneously
- Enables out-of-order execution => out-of-order completion
  - ID stage checks for hazards. If no hazards, issue the instn for execution. (Scoreboard ILP dates to CDC 6600 in 1963)

Key Ideas– ILP

- ILP is used in addition to pipelining.
- Processors with ILP are called *multiple-issue processors* – multiple instructions launched in 1 clock cycle. Two ways:
  - MIMD: Multiple Instructions Multiple Data
    - Very long instruction word (VLIW) – static multiple issue
    - Super-pipeline
    - Superscalar – dynamic multiple issue
  - SIMD: Single Instruction Multiple Data
    - Vector processor

Scalar Processors

Super-pipeline and Superscalar



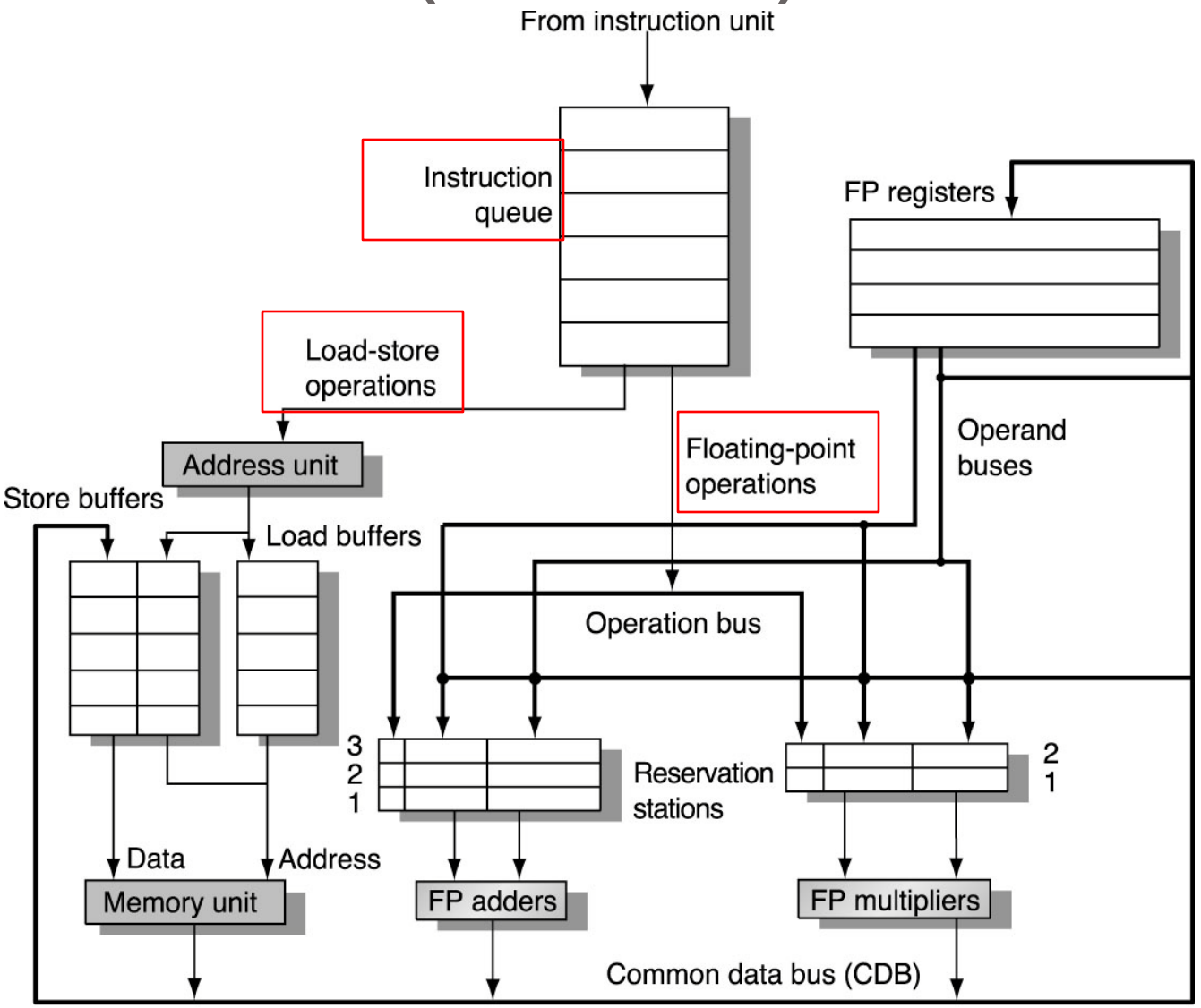
How ILP Works

- Issuing multiple instructions per cycle would require fetching multiple instructions from memory per cycle => called Superscalar degree or Issue width

- To find independent instructions, we must have a big pool of instructions to choose from, called instruction buffer (IB). As IB length increases, complexity of decoder (control) increases that increases the datapath cycle time

- Prefetch instructions sequentially by an Instruction Fetch Unit (IFU) that operates independently from datapath control. Fetch instruction (PC)+L, where L is the IB size or as directed by the branch predictor.

Microarchitecture of a Superscalar CPU (Power PC)



Microarchitecture of a Superscalar CPU (Intel Pentium 4)

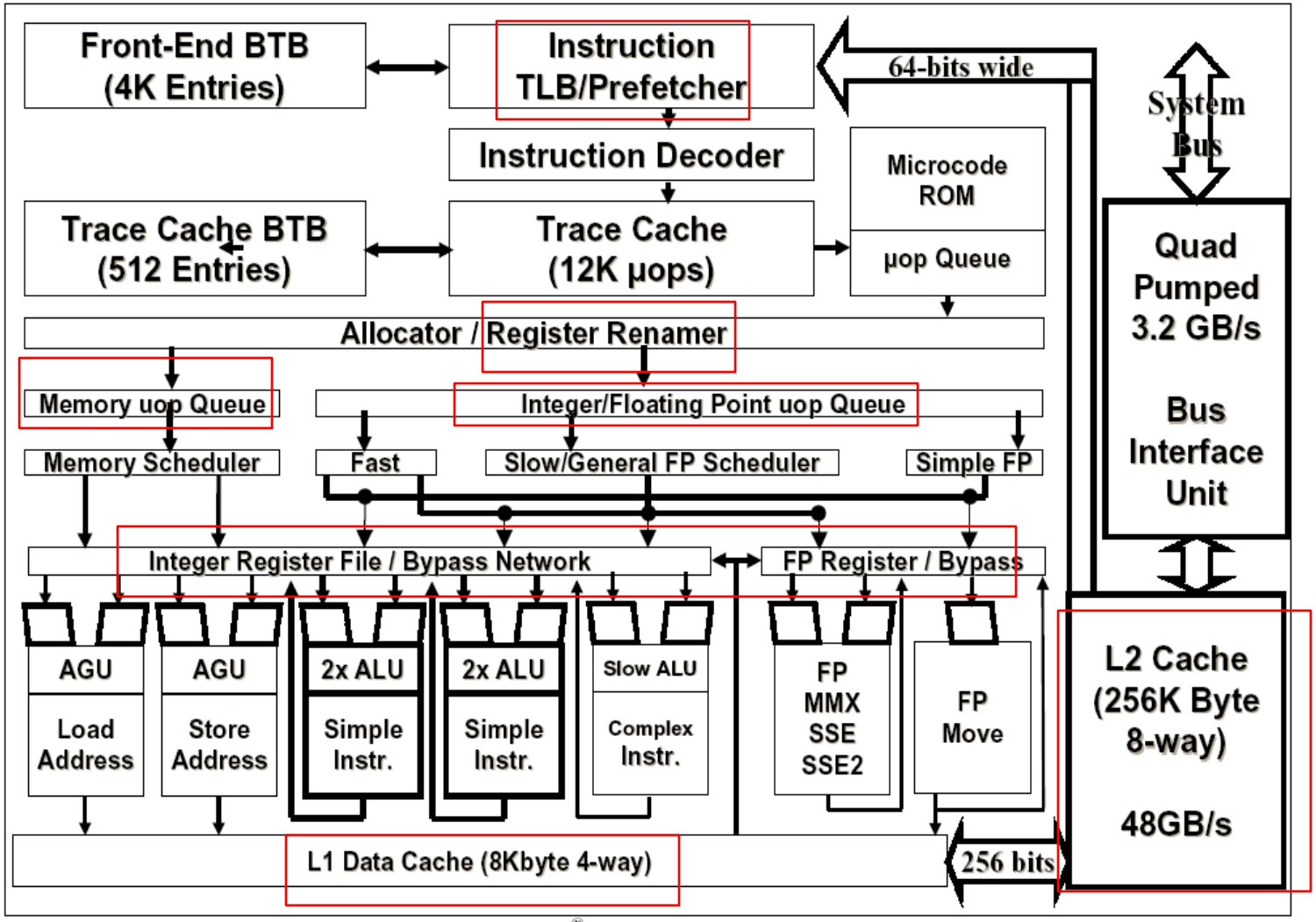


Figure 4: Pentium® 4 processor microarchitecture

A Static Two-Issue MIPS Pipeline

- ILP determined by Compiler
- Read two instructions per cycle
  - First : An ALU or branch instruction, and
  - Second : A load or store instruction
  - Insert one NOP if above pair is not possible for Exec
- Added hardware
  - A second instruction memory (Buffer/Queue)
  - Additional input/output ports in register file
  - Additional ALU in execute stage for address calculation

An Example

```
Loop:   lw      $t0, 0($s1)
        addu    $t0, $t0, $s2
        sw      $t0, 0(s1)
        addi    $s1, $s1, – 4
        bne     $s1, $0, Loop
```

With pipeline
CPI = 1
Total = 5

Instruction Queue/Buffer

Static Two-Issue Execution

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	nop	lw $t0, 0($s1)	1
	addi $s1, $s1, –4	nop	2
	addu $t0, $t0, $s2	nop	3
	bne $s1, $0, Loop	sw $t0, 4($s1)	4

Note code reordering to avoid pipeline stalls and change in sw argument (4) since addi now before sw nop since data dependency

CPI = 4/5 = 0.8 > 0.5 (ideal without nop)

With pipeline

With Loop Unrolling (Multiple of 4)

Loop Unrolling is essential for ILP Processors Why?
But, increase in Code memory and no. of registers.

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi $s1, $s1, –16	lw $t0, 0($s1)	1
	nop	lw $t1, 12($s1)	2
	addu $t0, $t0, $s2	lw $t2, 8($s1)	3
	addu $t1, $t1, $s2	lw $t3, 4($s1)	4
	addu $t2, $t2, $s2	sw $t0, 16($s1)	5
	addu $t3, $t3, $s2	sw $t1, 12($s1)	6
	nop	sw $t2, 8($s1)	7
	bne $s1, $0, Loop	sw $t3, 4($s1)	8

Note Register Change By Compiler

CPI = 8/14 = 0.57 > 0.5 (ideal)

With pipeline

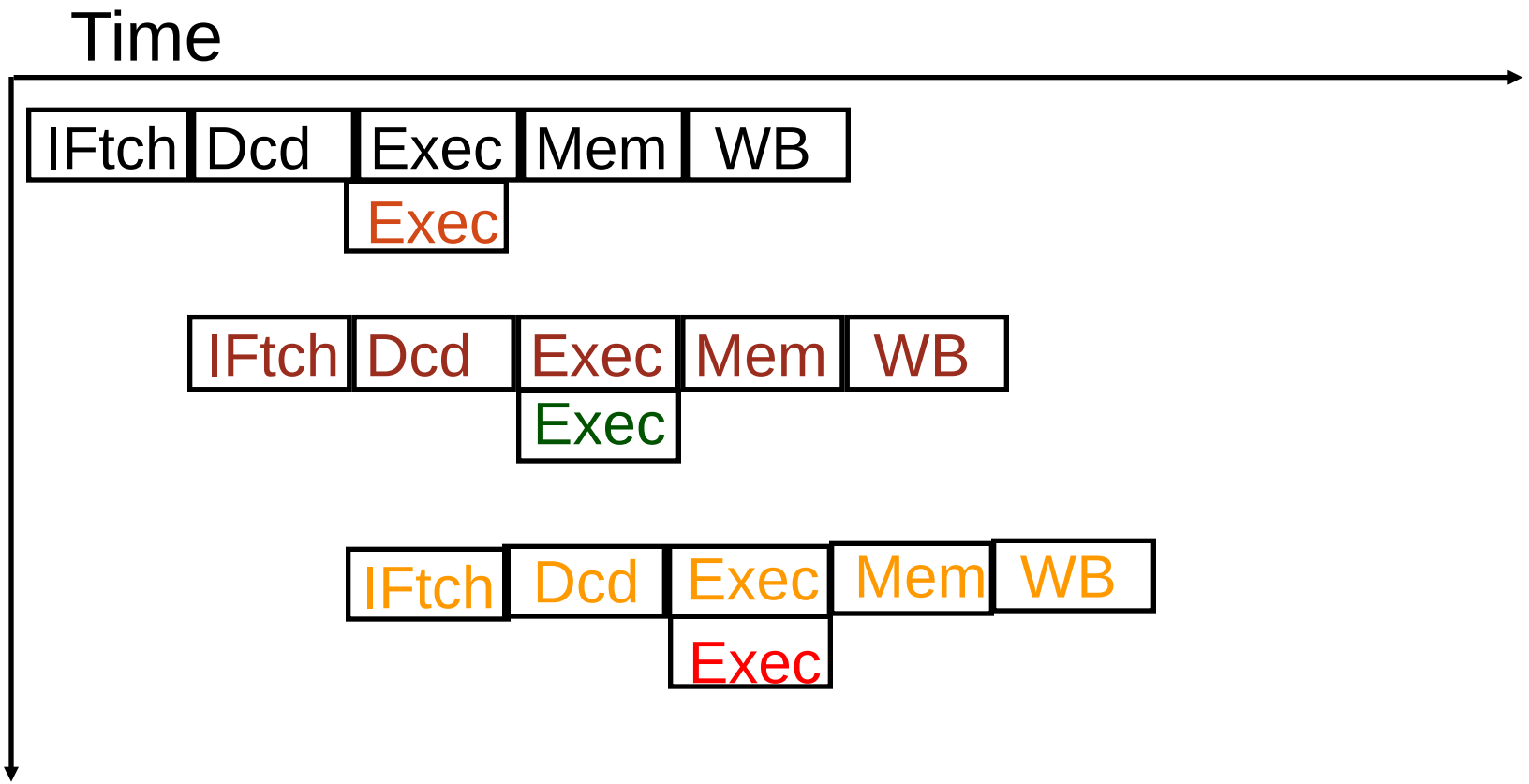
VLIW: Very Long Instruction Word

- Static multiple issue, ILP determined by compiler.
- Datapath contains multiple execution units.
- Compiler groups instructions that have no data or resource conflicts for parallel execution.
- Grouped instructions are packed in very long words of a wide instruction memory.
- Speedup benefit of VLIW is highly program dependent.

J. A. Fisher, "Very Long Instruction Word Architecture and ELI-512," *Proc. 10th Symp. on Computer Architecture*, Stockholm, June 1983, pp. 478-490.

J. A. Fisher, P. Faraboschi and C. Young, Embedded Computing: *A VLIW Approach to Architecture, Compilers and Tools*, Morgan Kaufmann.

Very Large Instruction Word (VLIW) IPC > 1, CPI <1



EX: Itanium

VLIW Architectures

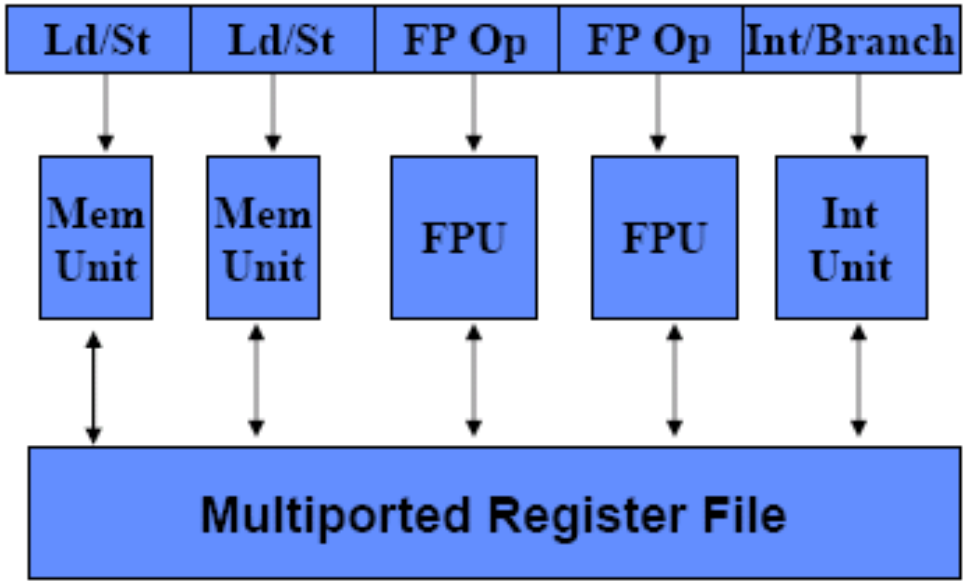
- Wide-issue processor that relies on compiler to
  - Packet together independent instructions to be issued in parallel
  - Schedule code to minimize hazards and stalls
- Very long instruction words (3 to 8 operations)
  - Can be issued in parallel without checks
  - If compiler cannot find independent operations, it inserts nops
- Advantage: simpler HW for multiple issue (ILP)
  - Faster clock cycle
- Disadvantages:
  - Code size
  - Requires aggressive compilation technology

VLIW and Superscalar

- Sequential stream of long instruction words
- Instructions scheduled statically by the compiler
- Number of simultaneously issued instructions is fixed during compile-time
- Instruction issue is less complicated than in a superscalar processor

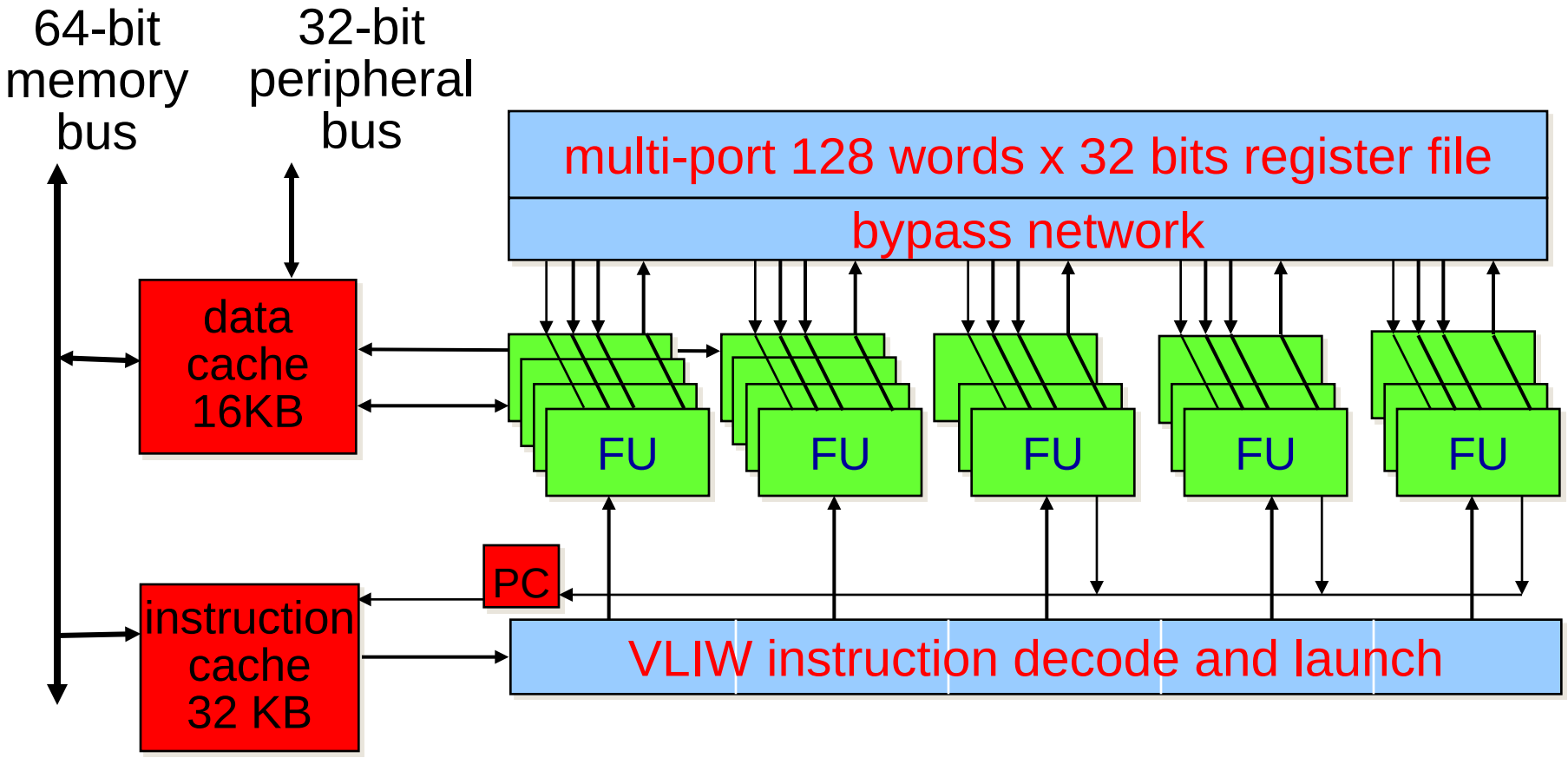
- Disadvantage: VLIW processors cannot react on dynamic events, e.g. cache misses, with the same flexibility like superscalars.
- Padding VLIW instructions with no-ops is needed in case the full issue bandwidth is not be met. This increases code size. More recent VLIW architectures use a denser code format which allows to remove the no-ops.
- VLIW is an *architectural* technique, whereas superscalar is a *microarchitecture* technique.

Traditional VLIW Hardware



- Multiple functional units, many registers (e.g. 128)
  - Large multiported register file (for N-FUs need ~3N ports)
- Simple instruction fetch unit
  - No checks, direct correspondence between slots & FUs
- Instruction format
  - 16 to 24 bits per op => 5*16=80 bits to 5*24=120 bits wide

TriMedia TM32 Architecture



Compressed code in the Instruction Cache

VLIW Code Example

for (i=0; i<100; i++) X[i] = X[i] + B;

- MIPS code

```
Loop:   l.d f2, 0(r1)
        add.d f4, f0, f2
        s.d f4, 0(r1)
        addui r1, r1, 8
        bne r1, r2, Loop
```

Unrolled Loop (7 times) by Compiler VLIW

#	Mem1	Mem2	FP1	FP2	Int/Branch
1	l.d f2, 0(r1)	l.d f6,8(r1)			
2	l.d f10,16(r1)	l.d f14,24(r1)			
3	l.d f18,32(r1)	l.d f22,40(r1)	add.d f4,f0, f0	add.d f8,f0, f6	
4	l.d f26,48(r1)		add.d f12,f0,f10	add.d f16,f0,f14	addu r1,r1,#56
5			add.d f20,f0,f18	add.d f24,f0,f22	
6	s.d f4,-56(r1)	s.d f8,-48(r1)	add.d f28,f0,f26		
7	s.d f12,-40(r1)	s.d f16,-32(r1)			
8	s.d f20,-24(r1)	s.d f24,-16(r1)			
9	s.d f28, -8(r1)				bneq r1,r2,Loop

- Assuming 2 cycle loads, 3 cycle FP adds (with pipeline)
- 9 cycles for 7 iterations
  - 1.28 cycles/iteration
  - 2.6 operations/cycle
- 21 nops (empty slots)
- 24 FP registers used (instead of just 3)

Problems with Early VLIW

- Characteristics
  - Large number of slots per long instruction
  - All operations in a long instructions must be independent
  - No interlocks (check for hazards across long instructions)
  - Fixed/direct correspondence between instruction slots and FUs
- Code size problems
  - Need many nops within each long instructions
  - Potential solution: code compression
- Binary compatibility problems
  - Different ISA, every time number or latency of FUs changes
  - Must Recompile for every processor generation
  - Potential solution: binary translation

Modern VLIW Processors

- Hardware interlocks
  - HW check for dependencies across LIWs
- Use a special field (bit mask) to determine if operations in LIW are independent or not
  - E.g. stop bits in IA-64
  - Can also specify dependence to following LIW
- Support several combinations of operation types per LIW
  - E.g., allow both Mem-FP-Int and Mem-Int-Int
  - Requires network that will "route" operations to proper FUs

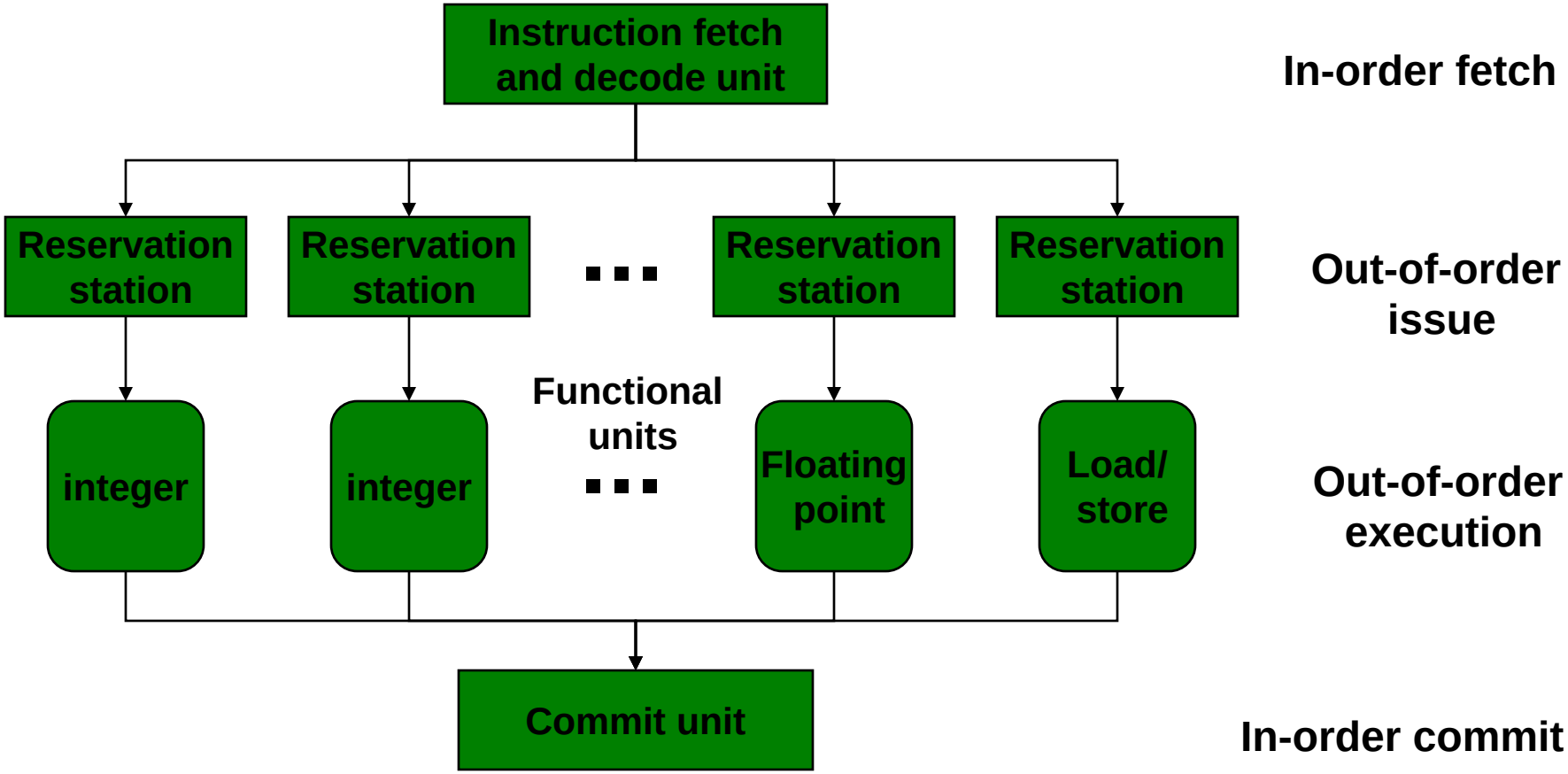
How Can the HW Help the Compiler with Discovering ILP?

- Compiler's performance is critical for VLIW processors
  - Find many independent instructions & schedule them in best possible way
- What limits the compiler's ability to discover ILP
  - Name dependencies (WAW & WAR)
    - Can eliminate with large number of registers
  - Branches
    - Limit compiler's ability to schedule
    - Modern VLIW processors use branch prediction too
  - Dependencies through memory
    - Force the compiler to use conservative schedule
- Can the HW help the compiler?
  - Superscalar processors

Superscalar Design of P4 (CISC)

- CISC shell:
  - Processor fetches instructions from memory in the order of static program.
  - Each instruction is translated into one or more fixed-length RISC instructions, known as micro-operations (micro-ops).
- RISC core (Execution):
  - Micro-ops are executed out-of-order in a dynamically scheduled pipeline.
  - Processor commits the result of each micro-op execution to register file in order of original program flow.

Superscalar: Dynamic Scheduling and Out-of-Order Execution



Superscalars

- 3 or more instruction issues per clock:
  - Intel P6
  - AMD K5
  - Sun UltraSPARC
  - Alpha 21164
  - MIPS R10000
  - PowerPC 604/620
  - HP 8000
- References:
  - D. W. Anderson, F. J. Sparacio and R. M. Tomasulo, "The IBM 360 Model 91: Processor Philosophy and Instruction Handling," *IBM J. Res. Dev.*, vol. 11, pp. 8-24, January 1967.
  - T. Agerwala and J. Cocke, "Reduced Instruction Set Processors," Technical Report RC12434 (#55845), Yorktown Heights, NY: IBM T. J. Watson Research Center, January 1987.

Out of Order Execution (OOE)

- A procedural programming language sequences instructions.
- Sequencing assumes an order of execution – no parallelism.
- OOE must preserve correctness of result.
- Principle: Two instructions can be executes in parallel if they do not have dependences.

RAW Dependence

- Read after write (RAW): A dependent instruction reads from a register being written to by another instruction.
- Example:

```
add   $s1, $s2, $s3
sub   $s2, $s1, $s3
```

sub has RAW dependence on add

WAR Dependence

- Write after read (WAR): A dependent instruction writes to a register being read by another instruction.
- Example:

```
add   $s1, $s2, $s3
sub   $s2, $s0, $s3
```

sub has WAR dependence on add

WAW Dependence

- Read after write (RAW): One instruction writes to a register to being written to by another instruction.
- Example:

```
add   $s2, $s2, $s3
.............
sub   $s2, $s1, $s3
```

sub has WAW dependence on add

Superscalar Instruction Issue

- Rules:
  - RAW dependence – If any operand is being written, do not issue.
  - WAR dependence – If the result register is being read, do not issue.
  - WAW dependence – If the result register is being written, do not issue.
- Scoreboard:
  - Cycle by cycle record of registers and execution units showing how many instructions are using them.
  - Example 1: In-order issue (next 2 slides).
  - Example 2: Out-of-order issue (3rd slide).

Dynamic Scheduling (Scoreboard)

- Consider an example:
  - First with in-order issue
  - Then with out-of-order issue
- Assume:
  - Up to two instructions are fetched in a cycle
  - Instruction register can hold two instructions
  - An Instruction is issued in decode cycle, or must wait until there is no RAW, WAR or WAW dependence
  - An instruction can retire two or three cycles after it is issued

Ck cycle	Inst #	Decoded	Issue Inst#	Retire Inst#	Reg. to read								Reg. to write							
					0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3 = R0 * R1	1		1	1										1				
	2	R4 = R0 + R2	2		2	1	1									1	1			
2	3	R5 = R0 + R1	3		3	2	1									1	1	1		
	4	R6 = R1 + R4	-		3	2	1									1	1	1		
3					3	2	1									1	1	1		
4		In Order Issue		1	2	1	1										1	1		
				2	1	1												1		
				3																
5		*******	4			1			1										1	
	5	R7 = R1 * R2	5			2	1		1										1	1
6	6	R1 = R0 – R2	-			2	1		1										1	1
7				4		1	1													1
8				5																
9		*******	6		1		1							1						
	7	R3 = R3 * R1	-		1		1							1						

In-order Issue scoreboard (Continued)

Ck cycle	Instr #	Decoded	Issue Inst#	Retire Inst#	Reg. to read								Reg. to write							
					0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
10					1		1							1						
11				6																
12			7			1		1								1				
	8	R1 = R4 + R4	-			1		1								1				
13						1		1								1				
14						1		1								1				
15				7																
16			8						2					1						
17									2					1						
18				8																

Out-of-order scoreboard (Next 2 Slides)

Questions?

- RAW dependence: Inst# 4 (R6 = R1 + R4) could not be issued until cycle 5. Should Inst# 5 (R7 = R1 * R2) wait in queue?
- Answer: No. Inst# 5 can be issued in cycle 3 as there is no register conflict (out-of-order issue).
- WAR dependence: Must the issue of Inst#6 (R1 = R0 – R2) waits until cycle 9 when all instructions reading R1 have retired?
- Answer: No. Provided new result of Inst#6 does not affect R1 being used by previous instructions (register renaming: R1⟶ R1*).

Ck cycle	Inst #	Decoded	Issue Inst#	Retire Inst#	Reg. to read								Reg. to write							
					0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3 = R0 * R1	1		1	1										1				
	2	R4 = R0 + R2	2		2	1	1									1	1			
2	3	R5 = R0 + R1	3		3	2	1									1	1	1		
	4	R6 = R1 + R4	-		3	2	1									1	1	1		
3	5	R7 = R1 * R2	5		3	3	2									1	1	1		1
	6	S1 = R0 – R2	6		4	3	3									1	1	1		1
				2	3	3	2									1		1		1
4			4		3	4	2		1							1		1	1	1
	7	R3 = R3 * S1	-		3	4	2		1							1		1	1	1
	8	S2 = R4 + R4	8		3	4	2		3							1		1	1	1
				1	2	3	2		3									1	1	1
				3	1	2	2		3										1	1
5				6		2	1		3					1					1	1
6			7			2	1	1	3					1		1			1	1
				4		1	1	1	2					1		1				1
				5				1	2					1		1				
				8				1								1				
7								1								1				
8								1								1				
9				7																

μ-architecture: Superscalar

Two types: Scoreboard and Tomasulo

Scoreboard (EX: PENTIUM):

- Out-of-order execution divides ID stage:
  1. Issue—decode instructions, check for structural hazards
  2. Read operands—wait until no data hazards, then read operands
- Scoreboards allow instruction to execute whenever there is no structural hazard or not waiting for prior instructions. So the instructions are issued in order, but can bypass the waiting instructions in the read operand stage => In-order issue Out-of-Order execution => Out-of-Order completion
- Named after CDC 6600 Scoreboard, which developed this capability

Scoreboard Implications

- Scoreboard replaces ID, EX, WB with 4 stages
- Out-of-order completion => WAR, WAW hazards?
- Solutions for WAR => Wait at the WB stage until the other instruction completes
- For WAW, must detect hazard at the ID stage: stall until other completes
- Need to have multiple instructions in execution phase => multiple execution units or pipelined execution units
- Scoreboard keeps track of dependencies, state or operations

CDC 6600 Scoreboard

- Speedup 1.7 from compiler; 2.5 by hand
  BUT slow memory (no cache) limits benefit
- Limitations of 6600 scoreboard:
  - No forwarding hardware
  - Limited to instructions in basic block (small *window*)
  - Small number of functional units (structural hazards), especailly integer/load store units
  - Do not issue on structural hazards
  - Wait for WAR hazards
  - Prevent WAW hazards

Summary of ILP + Scoreboard

- Instruction Level Parallelism (ILP) in SW or HW
- Loop level parallelism is easiest to see
- SW parallelism dependencies defined for program, hazards if HW cannot resolve
- SW dependencies/compiler sophistication determine if compiler can unroll loops
  - Memory dependencies hardest to determine
- HW exploiting ILP
  - Works when can't know dependence at run time
  - Code for one machine runs well on another
- Key idea of Scoreboard: Allow instructions behind stall to proceed (Decode => Issue instr & read operands)
  - Enables out-of-order execution => out-of-order completion
  - ID stage checked both for structural

Tomasulo Algorithm
(Implemented in IBM 360/91 in 1966)

- Control & buffers distributed with Function Units (FU) vs. centralized in scoreboard;
  - FU buffers called "reservation stations"; have pending operands
- Registers in instructions replaced by values or pointers to reservation stations(RS); called register renaming ;
  - avoids WAR, WAW hazards
  - More reservation stations than registers, so can do optimizations compilers can't
- Results to FU from RS, not through registers, over Common Data Bus that broadcasts results to all FUs
- Load and Stores treated as FUs with RSs as well

Tomasulo Summary

- Reservations stations: renaming to larger set of registers + buffering source operands
  - Prevents registers as bottleneck
  - Avoids WAR, WAW hazards of Scoreboard
  - Allows loop unrolling in HW
- Helps cache misses as well
- Lasting Contributions
  - Dynamic scheduling
  - Register renaming
  - Load/store disambiguation
- IBM 360/91 descendants are Pentium II; PowerPC 604; MIPS R10000; HP-PA 8000; Alpha 21264

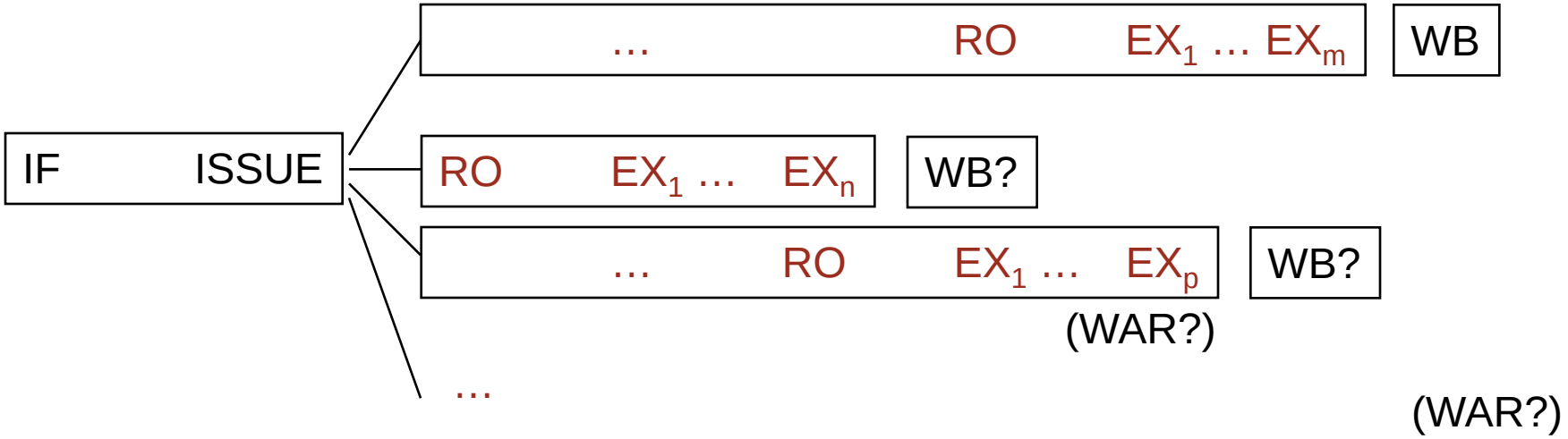
Tomasulo v. Scoreboard

IBM 360/91 v. CDC 6600

Pipelined Functional Units	Multiple Functional Units
(6 load, 3 store, 3 +, 2 x/÷)	(1 load/store, 1 + , 2 x, 1 ÷)
window size: ≤ 14 instructions	≤ 5 instructions
No issue on structural hazard	same
WAR: renaming avoids	stall completion
WAW: renaming avoids	stall completion
Broadcast results from FU	Write/read registers
distributed reservation stations	central scoreboard

HW Schemes: Out-of-order execution

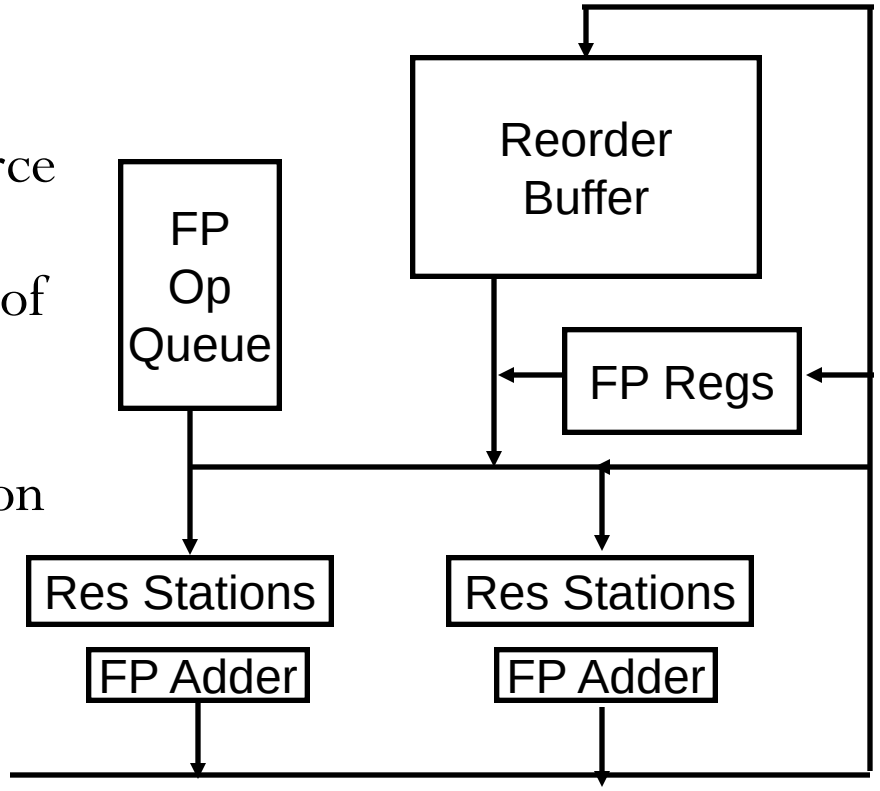
- Out-of-order execution divides ID stage:
  1. Issue—decode instructions, check for structural hazards, Issue in order if the functional unit is free and no WAW.
  2. Read operands (RO)—wait until no data hazards, then read operands
  - ADDD would stall at RO, and SUBD could proceed with no stalls.



- Scoreboards allow instruction to execute whenever 1 & 2 hold, not waiting for prior instructions.

HW support: Reorder Buffer

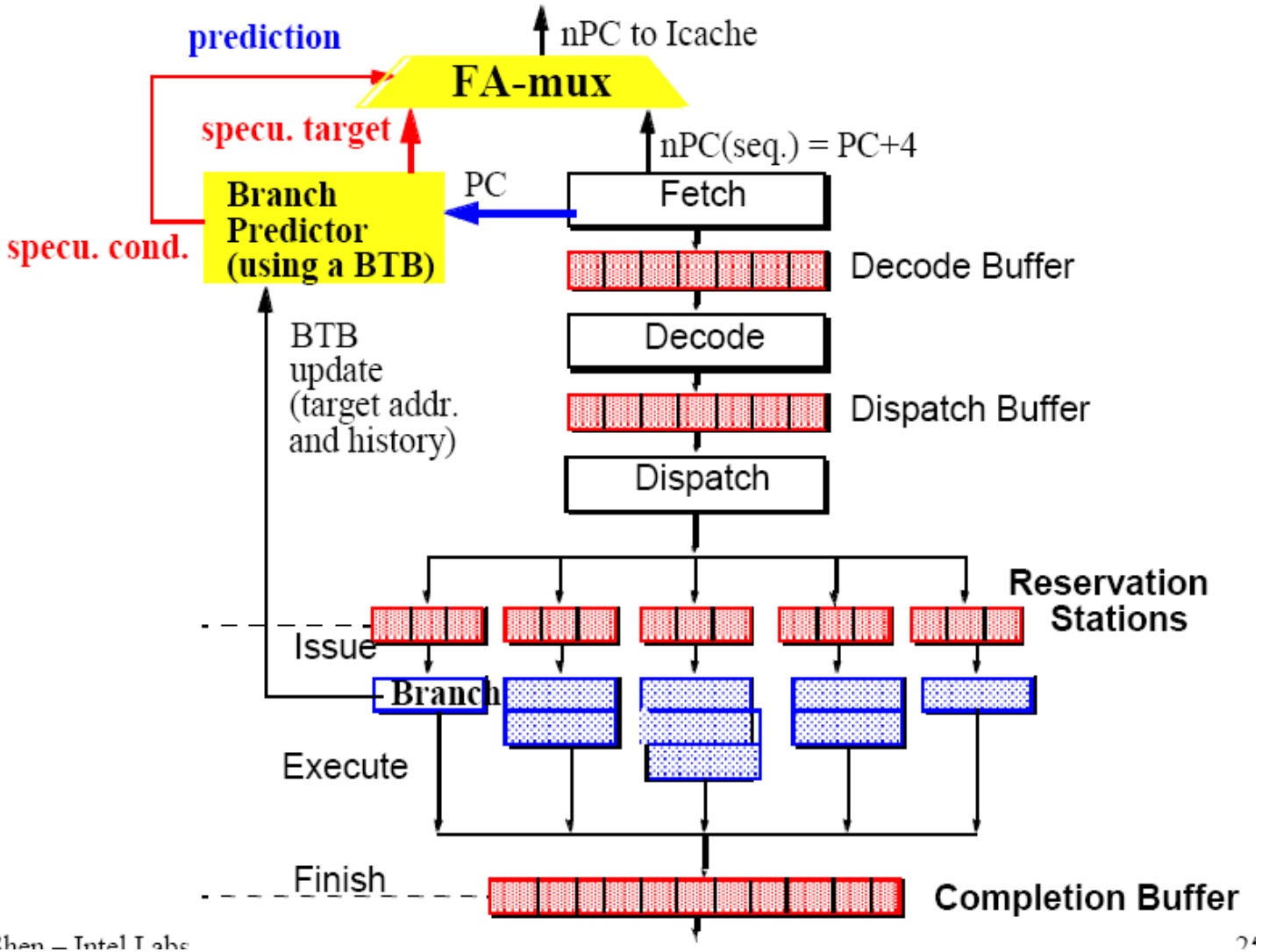
- Need HW buffer for results of uncommitted instructions: *reorder buffer*
  - 3 fields: instr, destination, value
  - Reorder buffer can be operand source => more registers like RS
  - Use reorder buffer number instead of reservation station when execution completes
  - Supplies operands between execution complete & commit
  - Once operand commits, result is put into register
  - Instructions commit in order
  - As a result, its easy to undo speculated instructions on mispredicted branches or on exceptions



HW support for More ILP

- ***Speculation***: allow an instruction to issue that is dependent on branch predicted to be taken *without* any consequences (including exceptions) if branch is not actually taken ("HW undo"); called "boosting" *branch predicted ++
- Combines branch prediction with dynamic scheduling to execute before branches resolved
- Separate *speculative* bypassing of results from real bypassing of results
  - When instruction no longer speculative, write boosted results (*instruction commit*) or discard boosted results
  - execute out-of-order but commit in-order to prevent irrevocable action (update state or exception) until instruction commits

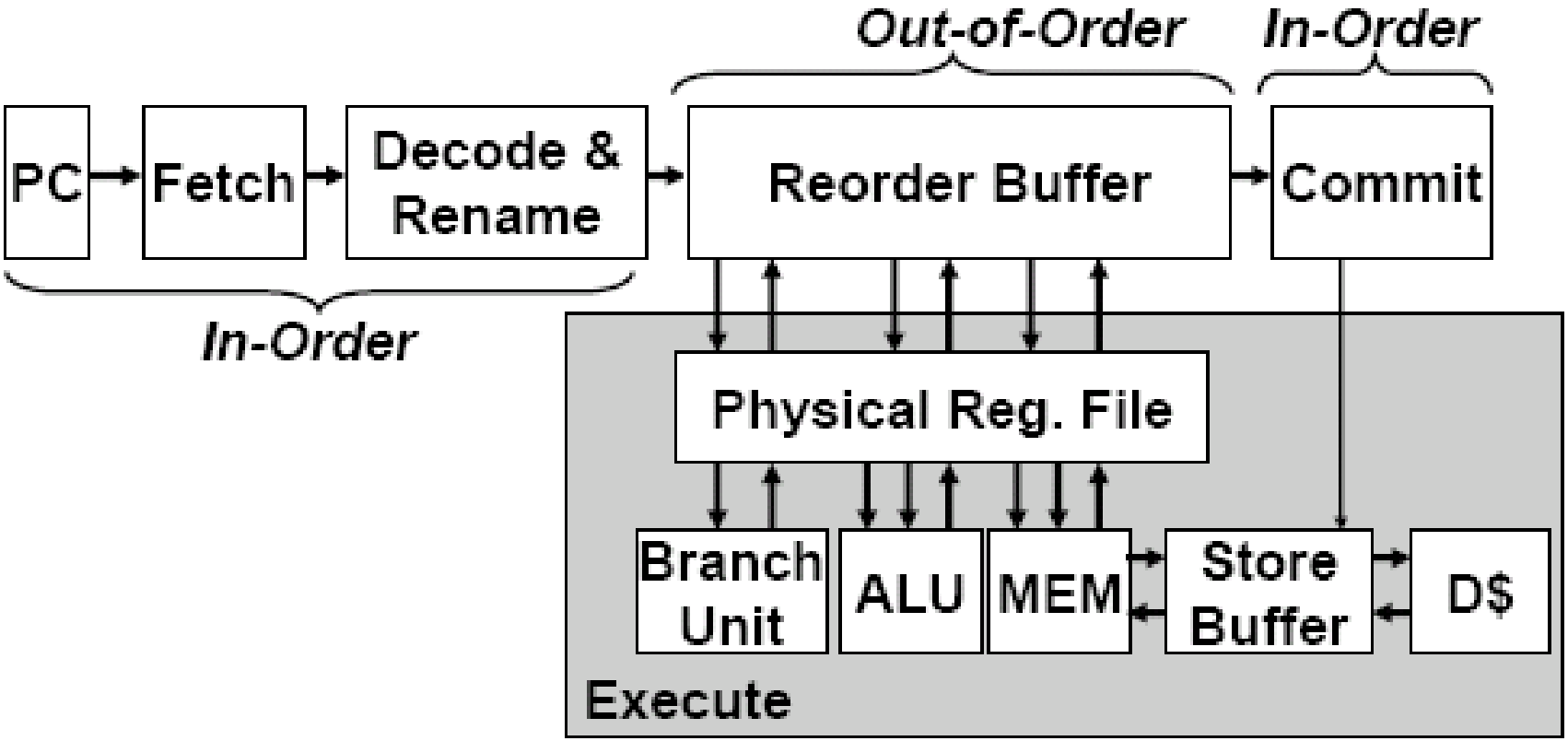
Branch Instruction Speculation



Renaming Registers

- Common variation of *speculative* design
- Reorder buffer keeps instruction information but not the result
- Extend register file with extra renaming registers to hold speculative results
- Rename register allocated at issue; result into rename register on execution complete; rename register into real register on commit
- Operands read either from register file (real or speculative) or via Common Data Bus
- Advantage: operands are always from single source (extended register file)

Speculative, Out-of-Order Superscalar Processor



Power Reduction by Slack Scheduling

- Application: Superscalar, out-of-order execution:
  - An instruction is executed as soon as the required data and resources become available.
  - A commit unit reorders the results.
- Delay the completion of instructions whose result is not immediately needed.
- Example of RISC instructions:
  - add r0, r1, r2; (A)
  - sub r3, r4, r5; (B)
  - and r9, r1, r9; (C)
  - or r5, r9, r10; (D)
  - xor r2, r5, r11; (E)

J. Casmira and D. Grunwald, "Dynamic Instruction Scheduling Slack," *Proc. ACM Kool Chips Workshop*, Dec. 2000.

Slack Scheduling